**Zpráva k projektu BI-VWM.21**

**Název projektu:** Agentic Hybrid RAG pro technickou podporu (StackOverflow Dataset)

**Seznam členů týmu:** Rustem Sandibekov (@sandirus) a Daria Krylova (@krylodar)

**1. Popis projektu**

**Cíl projektu:** Cílem projektu bylo vytvořit inteligentního Q&A (Question & Answering) asistenta, který dokáže odpovídat na komplexní programátorské dotazy na základě reálných dat z fóra StackOverflow. Systém využívá architekturu Agentic Hybrid RAG (Retrieval-Augmented Generation), která minimalizuje halucinace jazykových modelů tím, že jim poskytuje relevantní kontext z databáze.

**Vstupy projektu:** Uživatelský dotaz v přirozeném jazyce (např. *"How does the Python interpreter work?"*).

- Očištěný dataset otázek a odpovědí ze StackOverflow (cca 10 000 dokumentů pro studijní učely).

**Výstupy projektu:** * Relevantní odpověď vygenerovaná LLM (OpenAI) na základě vyhledaného kontextu.

- Seznam použitých zdrojů (zdrojové dokumenty ze StackOverflow), ze kterých model čerpal.

**2. Způsob řešení**

Projekt kombinuje dva odlišné přístupy k vyhledávání informací (Information Retrieval), které jsou řízeny autonomním agentem:

- **Lexikální vyhledávání (BM25):** Využívá přesnou shodu klíčových slov a frekvenční analýzu (TF-IDF/BM25 algoritmus). Je ideální pro hledání specifických chybových hlášek nebo názvů knihoven.
- **Sémantické (Vektorové) vyhledávání:** Uživatelský dotaz i dokumenty jsou převedeny do hustých vektorů (dense embeddings). Podobnost je počítána pomocí kosinové vzdálenosti (Cosine Similarity). Zvládá pochopit význam dotazu i při použití synonym.
- **Hybridní vyhledávání (Reciprocal Rank Fusion - RRF):** Kombinuje výsledky z obou metod. Skóre dokumentu se počítá podle vzorce:

$score = \frac{1}{k + rank_{BM25}} + \frac{1}{k + rank_{Vector}}$

Kde $k$ je vyhlazovací konstanta (typicky 60) a $rank$ je pozice dokumentu v daném vyhledávači.

- **Agentic Workflow:** Před samotným vyhledáváním "Query Rewriter" agent optimalizuje dotaz pro vyhledávače. Následně "Router" agent analyzuje dotaz a rozhodne, zda se použije BM25, Vector, nebo Hybrid režim.

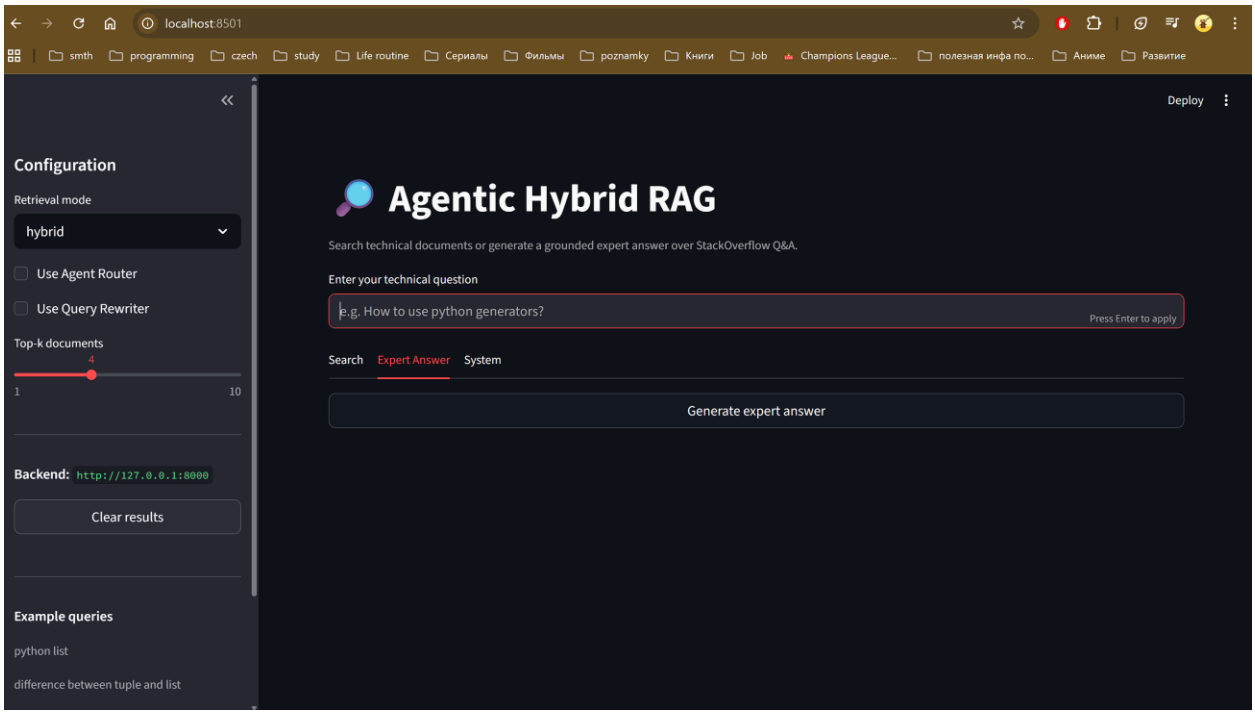
**3. Implementace**

- **Programovací jazyky:** Python 3.11
- **Architektura:** Mikroslužbová architektura kontejnerizovaná pomocí Docker Compose. Skládá se z backendu (API), frontendu a tří oddělených databází.
- **Databáze:** * PostgreSQL (úložiště primárních relačních dat).
    - Elasticsearch (BM25 lexikální index).
    - Qdrant (Vektorová databáze pro sémantické vyhledávání).
- **Klíčové knihovny:** FastAPI (backend), Streamlit (frontend), sentence-transformers (lokální generování embeddingů pomocí modelu BAAI/bge-small-en-v1.5), openai (LLM generátor), elasticsearch, qdrant-client.
- **Spuštění aplikace:** Projekt je plně přenositelný. Pro spuštění stačí nastavit OPENAI_API_KEY v konfiguraci a spustit příkaz docker-compose up --build.

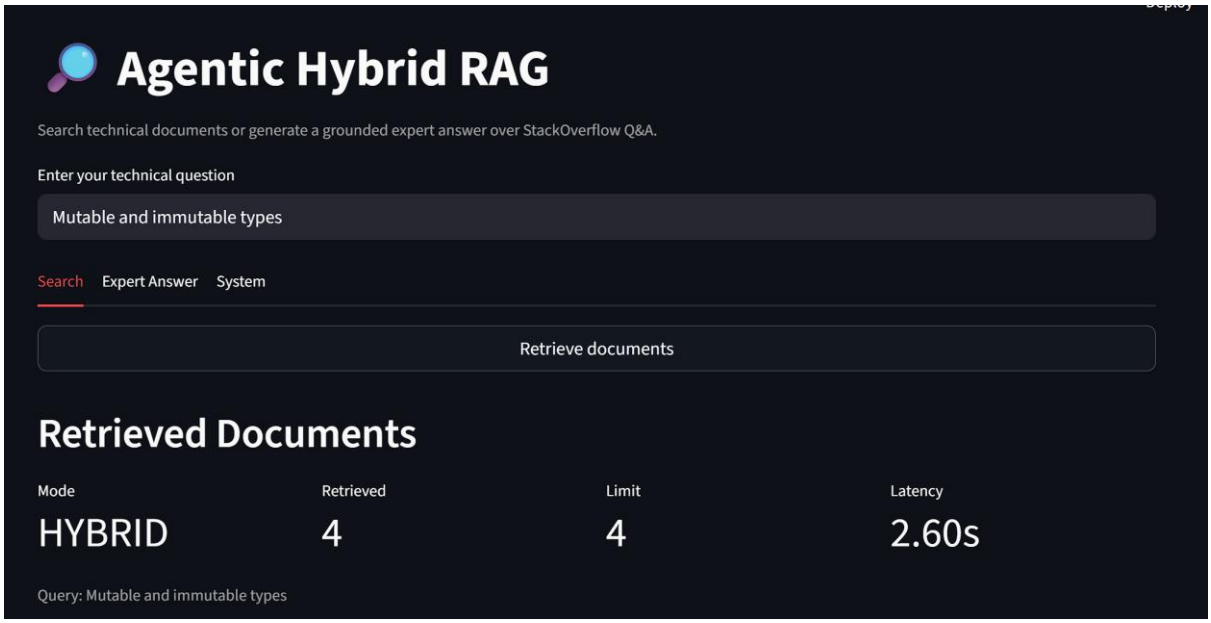
**4. Příklady vstupu a výstupu**

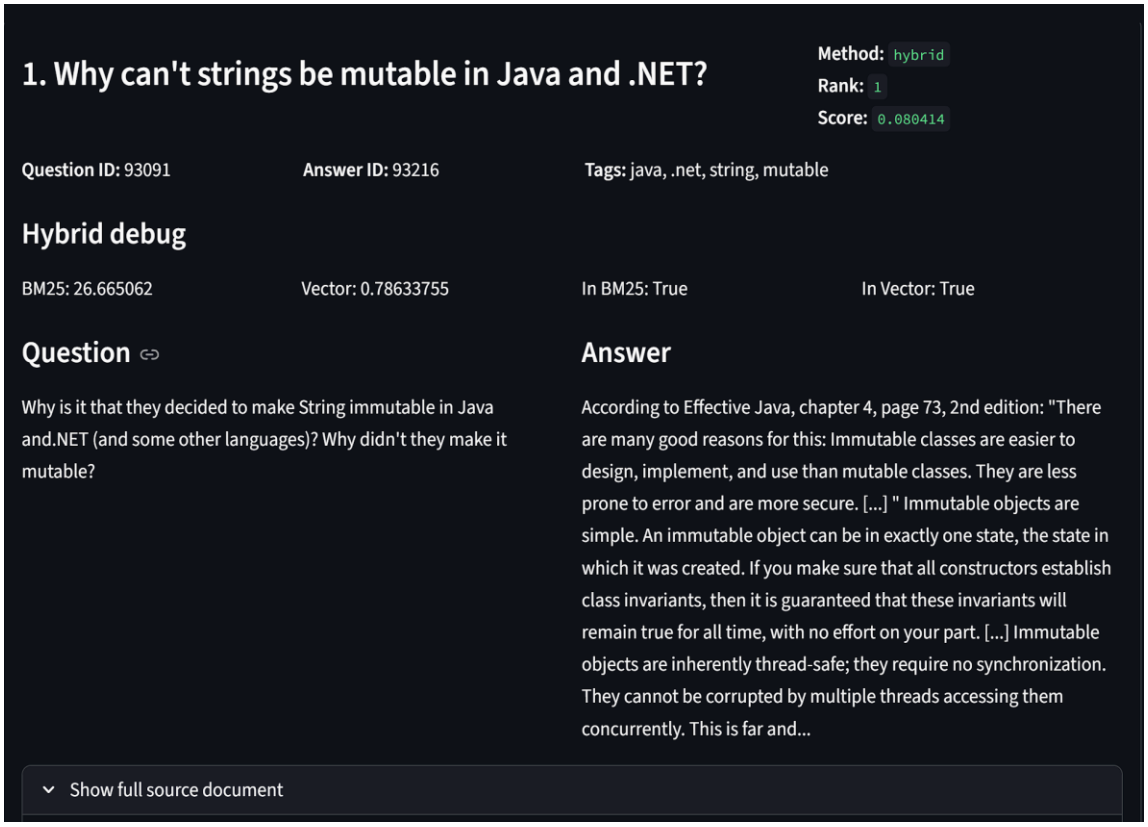
*(Здесь сделай 2-3 скриншота своего работающего Streamlit интерфейса. Подпиши их так:)*

- **Obrázek 1:** Uživatelské rozhraní aplikace s dotazem na formátování řetězců v Pythonu.




- **Obrázek 2:** Detail vygenerované odpovědi včetně odkazů na konkrétní zdrojové dokumenty (Groundedness).






**5. Experimentální sekce**

Byl proveden benchmark systému na testovací sadě 15 technických dotazů, aby se porovnal výkon a přesnost tří implementovaných režimů (BM25, Vector, Hybrid). Zkoumala se závislost latence na zvoleném algoritmu a kvalita odpovědí.

**Závislost rychlosti (Latence) na režimu vyhledávání:**

- **BM25:** Průměrně 2.88 s. Nejnižší latence, protože se spoléhá pouze na rychlý full-textový index.
- **Vector:** Průměrně 4.12 s. Zvýšení latence je způsobeno nutností prohnat uživatelský dotaz přes lokální embedding model.
- **Hybrid (RRF):** Průměrně 4.35 s. Nejnáročnější operace vyžadující paralelní dotazy do dvou databází a následný matematický re-ranking, avšak penalizace vůči čistě vektorovému hledání je minimální (cca +200 ms).

**Kvalitativní hodnocení (Correctness & Groundedness):**

Experiment ukázal, že lexikální přístup (BM25) často selhává, pokud dotaz neobsahuje přesná klíčová slova z dokumentu (systém odpoví "nedostatek kontextu"). Čistě vektorové vyhledávání zase někdy ztrácí preciznost u specifických IT termínů. Hybridní vyhledávání eliminovalo nedostatky obou metod a poskytlo fakticky nejsprávnější odpovědi bez halucinací.

**6. Diskuze**

Jelikož se jedná o *proof of concept*, projekt naráží na několik implementačních limitů, které by bylo nutné v produkčním prostředí řešit:

1. **API Rate Limits:** Během experimentální fáze systém narážel na restrikce OpenAI API (chyba 429 Too Many Requests a následně 500 Internal Server Error na backendu). Bylo nutné implementovat umělá zpoždění (time.sleep()). Řešením by bylo asynchronní zpracování front a exponenciální backoff (např. pomocí knihovny Tenacity).

2. **Hardwarová náročnost embeddingů:** Ingesce 10 000 dokumentů a tvorba sémantických vektorů probíhala na CPU, což trvalo značnou dobu. V produkci by bylo vhodné využít akceleraci na GPU (CUDA) a dávkové zpracování (batching).
3. **Automatická evaluace:** Větší testování by vyžadovalo nasazení automatizovaných metrik (tzv. LLM-as-a-Judge, např. framework Ragas), což však v tomto rozsahu omezoval rozpočet na API tokeny.

**7. Závěr**

Projekt úspěšně ověřil poznatky o moderním vyhledávání z přednášek v praxi. Podařilo se postavit funkční systém kombinující klasický BM25 algoritmus s moderním vektorovým vyhledáváním a LLM generací. Experimenty potvrdily, že architektura Agentic Hybrid RAG poskytuje nejrobustnější a nejpřesnější výsledky pro doménově specifické technické QA systémy, i za cenu mírně vyšší výpočetní latence.